

[Code](#)
[Issues](#)
[Pull requests](#)
[Actions](#)
[Projects](#)
[Wiki](#)
[Security](#)
[Insights](#)
[Settings](#)

[Preview](#) [Code](#) [Blame](#) 356 lines (356 loc) · 12.2 KB [Raw](#) [Copy](#) [Download](#) [Edit](#) [More](#)

 Open in Colab

In [4]:

```
# 1. Numpy arithmetic operation
import numpy as np
arr1=np.array([1,2,3,0])
arr2=np.array([4,1,0,6])
print(np.add(arr1,arr2))
print(np.multiply(arr1,arr2))
print(np.subtract(arr1,arr2))
print(np.divide(arr1,arr2))
try:
    if(arr2!=0):
        print(np.divide(arr1,arr2))
except:
    print("Divide by zero error")
```

```
[5 3 3 6]
[4 2 0 0]
[-3  1  3 -6]
[0.25 2.   inf 0. ]
Divide by zero error
```

```
<ipython-input-4-f547883315bc>:8: RuntimeWarning: divide by zero encountered in divide
print(np.divide(arr1,arr2))
```

In [25]:

```
# 2. arithmetic operation b/w 1D and 2D
import numpy as np
arr3=np.random.randint(1,100,5)
#if(arr3.reshape(4,5)):
    # print("ValueError: cannot reshape array of size 5 into shape (4,5)")
    #assign arr3=np.random.randint(1,100,20)
arr5=np.arange(20).reshape(4,5)
arr4=np.random.randint(1,100,(4,5))
print(np.add(arr5,arr4))
print(np.subtract(arr5,arr4))
```

```
[[ 59  19  76  66  94]
 [ 90   9  24  66  85]
 [ 21  13  95  20  75]
 [ 10  10  10  10  10]]
```

```

[[ 83  59  96 103  41]]
[[-59 -17 -72 -60 -86]
 [-80   3 -10 -50 -67]
 [ -1   9 -71  -4 -47]
 [-53 -27 -62 -67  -3]]

```

In [47]:

```

# 3. indexing and slicing
arr1=np.random.randint(1,27,(3,3,3))
print(arr1)
print("The 3rd column of 2nd 2-D array is")
print(arr1[1,:,2])
print("1st and 3rd row of 1st 2-D array is")
print(arr1[0,[0,2],:])
print("intersection of 1st row and third column in third 2-D array is")
print(arr1[2,0,2])

```

```

[[[ 6  2  6]
 [20  1 11]
 [18  5 13]]

```

```

[[22 20 17]
 [10 12 25]
 [20  5 13]]

```

```

[[25 23  8]
 [16  2 18]
 [22 22  4]]]

```

The 3rd column of 2nd 2-D array is

[17 25 13]

1st and 3rd row of 1st 2-D array is

```

[[ 6  2  6]
 [18  5 13]]

```

intersection of 1st row and third column in third 2-D array is

8

In [63]:

```

# 4. 1 to 100 with shape 5*4. Now perform following operations
import numpy as np
arr_2d=np.random.randint(1,100,(5,4))
print(arr_2d)
arr1=arr_2d[arr_2d%2==0]
print("Even numbers in an array is",arr1)
arr2=arr_2d[arr_2d%2!=0]
print("Odd numbers in an array is",arr2)

```

```
print( odd numbers in an array is ,arr2)
print("Row wise sum is", np.sum(arr_2d,axis=1))
print("Column wise sum is", np.sum(arr_2d,axis=0))
print("2-D into 3-D array is\n",arr_2d.reshape(2,2,5))
```

```
[[94 77 60 32]
 [52 13 94 89]
 [54  4 29 89]
 [44 22 87 55]
 [64 80 63 92]]
```

Even numbers in an array is [94 60 32 52 94 54 4 44 22 64 80 92]

Odd numbers in an array is [77 13 89 29 89 87 55 63]

Row wise sum is [263 248 176 208 299]

Column wise sum is [308 196 333 357]

2-D into 3-D array is

```
[[[94 77 60 32 52]
  [13 94 89 54  4]]
```

```
[[29 89 44 22 87]
 [55 64 80 63 92]]]
```

In [78]:

```
# 5. 1 to 100 with shape 5*4. Now perform following operations
a_2d=np.random.randint(1,100,(5,4))
print(a_2d)
a_2d[[0,2]]=a_2d[[2,0]]
print("After swap 1st row with 3rd row\n", a_2d)
a_2d[:,[1,3]]=a_2d[:,[3,1]]
print("After swap 2nd column with 4th column is\n",a_2d)
a_2d[a_2d < 50]=0
print(" Replace the values less than 50 with zero.\n",a_2d)
print("Convert this 2D array into 1-D array column wise\n",a_2d.flatten(order='F'))
print("Convert this 2D array into 1-D array row wise\n",a_2d.flatten(order='C'))
```

```
[[56 24 24 12]
 [28 82  5 36]
 [29 83 77 85]
 [71 15 78 35]
 [11 98  7 21]]
```

After swap 1st row with 3rd row

```
[[29 83 77 85]
 [28 82  5 36]
 [56 24 24 12]
 [71 15 78 35]
```

```

[[11 98 7 21]]
After swap 2nd column with 4th column is
[[29 85 77 83]
 [28 36 5 82]
 [56 12 24 24]
 [71 35 78 15]
 [11 21 7 98]]
Replace the values less than 50 with zero.
[[ 0 85 77 83]
 [ 0 0 0 82]
 [56 0 0 0]
 [71 0 78 0]
 [ 0 0 0 98]]
Convert this 2D array into 1-D array column wise
[ 0 0 56 71 0 85 0 0 0 0 77 0 0 78 0 83 82 0 0 98]
Convert this 2D array into 1-D array row wise
[ 0 85 77 83 0 0 0 82 56 0 0 0 71 0 78 0 0 0 0 98]

```

In []: *# 6. Stacking Explanation*

Stacking in NumPy refers to joining arrays along a new axis. It's a way to combine multiple arrays into a single, larger array. NumPy provides several functions for stacking, including:

np.stack: Joins arrays along a new axis. np.vstack: Stacks arrays vertically (row-wise). np.hstack: Stacks arrays horizontally (column-wise). np.dstack: Stacks arrays along the third axis (depth-wise). np.column_stack: Stacks 1-D arrays as columns into a 2-D array. np.row_stack: Stacks arrays vertically (row-wise). (Similar to np.vstack)

```
import numpy as np
```

Create sample arrays

```
a = np.array([1, 2, 3]) b = np.array([4, 5, 6])
```

np.stack - Joins arrays along a new axis

```
stacked_array = np.stack((a, b), axis=0) # Stack along axis 0 (creates a new row)
print("\n stack (axis=0)\n" stacked_array)
```

```
stacked_array = np.hstack((a, b)) # Stack along axis 0 (creates a new row) print("np.hstack (axis=0):\n", stacked_array)
```

```
stacked_array = np.stack((a, b), axis=1) # Stack along axis 1 (creates a new column) print("\nnp.stack (axis=1):\n", stacked_array)
```

np.vstack - Stacks arrays vertically (row-wise)

```
vstacked_array = np.vstack((a, b)) print("\nnp.vstack:\n", vstacked_array)
```

np.hstack - Stacks arrays horizontally (column-wise)

```
hstacked_array = np.hstack((a, b)) print("\nnp.hstack:\n", hstacked_array)
```

np.dstack - Stacks arrays along the third axis (depth-wise)

```
dstacked_array = np.dstack((a, b)) print("\nnp.dstack:\n", dstacked_array)
```

np.column_stack - Stacks 1-D arrays as columns into a 2-D array

```
column_stacked_array = np.column_stack((a, b)) print("\nnp.column_stack:\n", column_stacked_array)
```

Stacking vs. Concatenation

While both stacking and concatenation combine arrays, they do so in slightly different ways:

1. Dimensionality

Stacking: Introduces a new dimension along which the arrays are joined. The output array will have one more dimension than the input arrays.

Concatenation: Joins arrays along an existing dimension. The output array will have the same number of dimensions as the input arrays.

2. Axis of Operation:

Stacking: You specify the axis argument to determine the position of the new dimension where the arrays are stacked.

Concatenation: You specify the axis argument to determine along which existing axis the arrays are joined.

3. Shape Requirements:

****Stacking:** Input arrays must have the same shape (except along the stacking axis). **Concatenation:** Input arrays can have different shapes along the concatenation axis, but their other dimensions must be compatible.